Blink Qt — WebRTC video conferencing

How Blink Qt joins SylkServer conference rooms with audio and video over WebRTC, next to the MSRP chat it always had, and how it joins the rooms of other SylkServer domains.

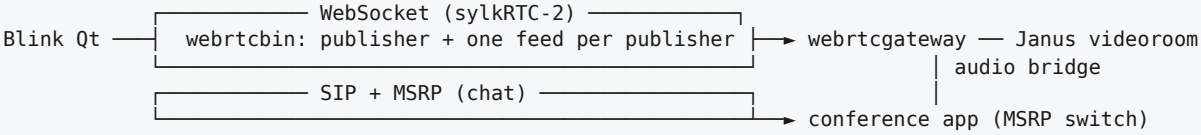
Code	`blink/webrtc/` (`__init__.py`, `sylkrtc.py`, `videoroom.py`, `videoroomwindow.py`, `sharedialog.py`); hooks in `sessions.py`, `chatwindow.py`, `mainwindow.py`, `__init__.py`, `logging.py`
Server side	SylkServer webrtcgateway (sylkRTC protocol, Janus videoroom), conference application (MSRP switch)
Related	`sylkserver` (`[MSRP] port`), `opensips-config-sylkserver` (`CONFERENCE_SERVER_PUBLIC`, Contact rewrite), `sylk-suite` (MSRP port, TLS certificate, source mode)
State	2026-10-24, first working version: own domain and a foreign domain (`domain.sylk.link`) tested

1. Overview

A SylkServer room is two things that share one name:

- the **video room**: a Janus videoroom (an SFU) behind the webrtcgateway, reached over a WebSocket with the sylkRTC protocol. Every participant publishes its camera and microphone on one PeerConnection and subscribes to each other publisher on one PeerConnection per publisher (a *feed*);
- the **MSRP switch**: the conference application, reached over SIP, carrying the room's chat (MSRP), its conference-info (participants) and, for SIP callers, the mixed audio. The gateway's audio bridge (`sylk-janus-audio-bridge`) joins the two.

Blink Qt joins the video room for **audio and video** and the MSRP switch for **chat only**. Sending audio on both would loop the room's audio through it; the MSRP window therefore never adds audio while a video room is joined (§5).



2. Loading and availability

Nothing of `blink/webrtc` beyond the package's `__init__.py` is imported until a SylkServer is detected: `__init__.py` holds the light checks (`ws_server`, `conference_video_available`, `room_video`, `active_videoroom`, …) and imports `videoroom` (GStreamer) and `sylkrtc` (Qt WebSockets) only after them.

Check	Where	Meaning
`ws_server(account)`	`__init__.py`	the `wsServer` of the account's domain, from SylkServer discovery (`_sylkserver.<domain>` TXT → configuration JSON)
`media_missing()`	`videoroom.py`	the missing pieces, as (what, Debian package) pairs
`media_available()`	`__init__.py`	`media_missing()` empty; checked once a run
`conference_video_available(account)`	`__init__.py`	both of the above

Required: Qt WebSockets, GStreamer introspection (`Gst`, `GstWebRTC`, `GstSdp`) and the elements `webrtcbin nicesrc appsrc appsink decodebin videoconvert videoscale videorate audioconvert audioresample audiomixer volume opusenc rtpopuspay autoaudiosrc autoaudiosink`, plus one video encoder and payloader pair. Optional: `webrtcdsp` + `webrtcechoprobe` (echo cancellation).

Debian packages (Recommends of `blink`): `gstreamer1.0-plugins-bad`, `gstreamer1.0-nice`, `gstreamer1.0-pulseaudio`, `gir1.2-gst-plugins-bad-1.0`, `python3-pyqt6.qtwebsockets`.

The Activity log says what was found, once per change of account, server or missing set:

```
[webrtc] Video conferences available for ag@sylk.link through wss://…/webrtcgateway/ws
[webrtc] WebRTC media: GStreamer 1.24.2, video VP9 (vp9enc), VP8 (vp8enc), H264 (x264enc), audio OPUS, e
cho cancellation webrtcdsp, Qt WebSockets yes
[webrtc] Video conferences for … need: nicesrc (install gstreamer1.0-nice)
```

3. Signalling: `sylkrtc.py`

`SylkRTCConnection` is one `QWebSocket` per `wsServer` URL (`SylkRTCConnections`), in the GUI thread, sub-protocol `sylkRTC-2`.

Constant	Value	
`CONNECT_TIMEOUT`	15 s	from opening to `ready-event`
`REQUEST_TIMEOUT`	20 s	a request without `ack`/`error` fails
`PING_INTERVAL`	30 s	WebSocket ping

- `request(data, callback)` adds a `transaction`, queues until ready, calls `callback(error)` on `ack` (None) or `error` (the reason).
- Events (`account-event`, `session-event`, `videoroom-event`) go out by the `received` signal (not `event`, which would shadow `QObject.event()`).
- `add_account(account)` / `add_identity(id, ha1, display_name)`: `account-add` with the HA1 of the SIP credentials, as `sylkrtc.js` sends it. The account is **never registered** through the gateway: Blink registers over SIP itself, a second registration would fork calls and messages to the gateway.
- TLS certificate errors are ignored when SIP TLS verification is off (`tls.verify_server`).
- A connection nobody uses is closed one second after its last room ends.

4. Media: `videoroom.py`

4.1 Session flow

```
account-add ─► build publisher, create offer ─► videoroom-join(offer)
  ◄─ session-state progress / accepted(answer) ─► link the video encoder, set answer
  ◄─ initial-publishers / publishers-joined ─► per publisher: videoroom-feed-attach
  ◄─ feed-attached(offer) ─► feed webrtcbin, answer ─► videoroom-feed-answer
  ◄─ session-state established, feed-established
leave: videoroom-leave
```

ICE candidates go out as `videoroom-session-trickle` (end of candidates: an empty list), held back until the request that creates their session on the server (join, feed answer) has been sent.

`PUBLISHER_TIMEOUT` (20 s from the offer) ends a join that never reaches *established*.

4.2 Pipeline

One GStreamer pipeline per room, so the echo canceller sees what is played:

```
camera (SDK frames) ─► appsrc ─► convert/scale/rate (640x480, 24 fps, I420) ─► encoder ─► payloader ─┐
microphone ─► webrtcdsp ─► volume (mute) ─► opusenc ─► rtpopuspay ──────────────────────────────────┤
webrtcbin (publisher)                                                                               ▼

webrtcbin (feed) ─► decodebin ─┬─► video: convert ─► appsink (BGRx) ─► the tile
                               └─► audio: convert/resample ─► audiomixer ─► webrtcechoprobe ─► speakers
```

- **Camera**: frames of the SDK's camera (`SIPApplication.video_device`) read through a `FrameBufferVideoRenderer`, like any other view of it, so the camera chosen in Blink, the screen included, is what the room sees. "Camera off" sends black frames.
- **Codecs**: the room's video codec is set on the server (Janus `videocodec`), so the offer lists every codec that can be encoded here (VP9 98, VP8 96, H264 102 with x264enc or openh264enc) and the encoder for the one the answer picked is linked afterwards; the transceiver's codec preferences are then narrowed to that codec (the encoder's output would not match the offered `profile-level-id`). Audio: Opus 111, `OPUS/48000/2`, given as codec preferences because the offer is built before the microphone has caps.
- **Bitrate**: 1024 kbit/s video, 32 kbit/s Opus with in-band FEC.
- **No GLib main loop**: the bus is read with a sync handler; everything that touches the signalling or the UI is handed to the GUI thread.
- **Orientation (CVO)**: a mobile sends the camera as it is held and says how it is turned in the RTP header extension `urn:3gpp:video-orientation`. Its id is taken from each feed's offer and added to the answer (webrtcbin drops extensions it does not implement); a pad probe reads it from the RTP packets and the decoded picture is turned clockwise by 0/90/180/270°.

4.3 Participants

`initial-publishers` / `publishers-joined` / `publishers-left` / `publisher-updated`.

Publisher type	Subscribed	Tile
`webrtc`	yes	picture and name
`bridge` (the audio bridge: the SIP callers' mixed audio)	yes (audio)	none
`sip` (a SIP caller behind the bridge)	no	initial and "audio"
`sip` with our own address (our MSRP chat leg)	no	none (`is_self`)

A moderator's `mute-request` mutes the microphone.

4.4 Room chat over WebRTC

`send_chat_message()` (`videoroom-message`) and the `message` / `message-delivery` events, posted as `VideoroomSessionGotMessage` / `VideoroomSessionDidDeliverMessage`. The gateway relays them to the room's MSRP switch. Used by the MSRP window when its own MSRP session to the room is not connected (§5.3).

4.5 Notifications

Name	Data
`VideoroomSessionDidChangeState`	state, reason, room
`VideoroomSessionAudioMutedDidChange`	muted, room

Name	Data
`VideoroomSessionGotMessage`	type, content, content_type, sender_uri, sender_name, timestamp
`VideoroomSessionDidDeliverMessage`	message_id, delivered, code, reason

5. User interface

5.1 Join Conference panel (`sessions.ConferenceDialog`)

- Media line: **Chat** (always checked, not changeable), **Audio**, **Video**. Video is shown only when the account's domain runs SylkServer and the media can be done here.
- **Video** checked: audio and video over WebRTC (Audio unchecked: the microphone starts muted), chat over MSRP. **Audio** only: SIP audio and chat to the bridge, as before.
- The room field takes a room name, `room@host` or a link `https://host/conference/room`. Rooms of another domain are looked up while typing, and **Join waits for the answer** (§6.2).
- Bottom line: "Conferencing provided by SylkServer at "; without SylkServer the original MSRP switch (RFC 7701) note; with missing packages, the `sudo apt install …` line.
- History: the rooms joined, most recent first, kept in `ui.conf` (`conference_dialog/recent_rooms`, 20).
- Invitees: REFERred through the MSRP session, or through the video room (`videoroom-invite`) when there is none.

A Video Call to a room contact joins the same way (WebRTC + MSRP chat). An audio call to a room whose video room is open joins with chat only.

5.2 Video conference window (`videoroomwindow.py`)

- Tiles in the grid that makes them largest at 16:9 (§7); our camera mirrored as "Me"; a short last row is centred; pictures fitted, not cropped.
- Call bar (the video call window's): **Mute**, **Camera off**, **Share**, **Full screen**, **Leave**. It hides after 5 s without mouse movement and comes back when the mouse moves anywhere in the window.
- Window title: `Room <name> · <n> participants · mm:ss` once connected; a status pill only while connecting and to say why the room ended.
- Listed in the main window's **Window** menu while open (restores it when minimized).
- **Share** opens the share dialog (§5.4).

5.3 MSRP window, for a room with a video room

Control	Behaviour
Header Chat / Audio / Video	only start what is not there, or bring up what is; pressed while it lasts. Audio and Video bring up the video window when a video room is open
Header Video	joins with audio and video over WebRTC, removing the session's SIP audio (or hanging up an audio-only session) first; shown for rooms that can have video, a foreign SylkServer room included even without `isfocus`
Header Share	the share dialog (§5.4), for rooms with a SylkServer configuration
Microphone	mutes the video room, in step with its window and a moderator
Add audio / Start audio and MSRP chat	not offered while a video room is open
Hang up	ends the chat and leaves the video room (its window closes)
Tile ×	in a room: hangs up (video room too), the tile stays; disconnected: closes the tile

Control	Behaviour
Input bar	enabled when the MSRP chat is connected, or when the video room is (the chat then goes over WebRTC)
Participants list	dropping contacts invites them: REFER over MSRP, else through the video room; the audio bridge is not listed

Room chat over WebRTC: when the MSRP chat is not connected, messages go out with `videoroom-message` and those of the room come in through the same tile, kept in history like MSRP ones (`ChatStreamGotMessage` / `ChatStreamWillSendMessage` with a stand-in stream); an id made of sender, time and content keeps the backlog the room replays from being stored twice. When the MSRP chat is connected it carries the chat and the WebRTC copies are ignored.

5.4 Share dialog (`sharedialog.py`)

As Sylk Mobile's ShareConferenceLinkModal, from the configuration of the room's own domain:

- Web: `<publicUrl>/conference/<room>`
- SIP: `<room>@<conference domain>` (the MSRP switch address)
- PSTN access number and room number, when `conference.pstnBridge` is set
- QR code of `sylk://<domain>/conference/<room>` (python3-qrcode or segno)

Each address can be selected and copied; Copy puts all of them on the clipboard, E-mail… opens a message with them.

6. Connecting to foreign domains

6.1 What a foreign room is

A room whose host is neither the account's conference server nor in its domain, given as `room@conference.example.com`, `room@example.com` or a link `https://example.com/conference/room`. Its SylkServer is found by the same discovery as for accounts, for the **room's** domain: `conference.` and `videoconference.` are stripped from the host and `_sylkserver.<domain>` is looked up.

Example, `https://domain.sylk.link/conference/others4`:

From	Value
TXT `_sylkserver.domain.sylk.link`	`https://domain.sylk.link/sylk-config.json`
`wsServer`	`wss://domain.sylk.link/ws`
`defaultConferenceDomain`	`videoconference.domain.sylk.link` → video room `others4@videoconference.domain.sylk.link`
MSRP switch (convention)	`others4@conference.domain.sylk.link` (`videoconference.` → `conference.`)
`iceServers`	that server's (here STUN only)
`defaultGuestDomain`	`guest.domain.sylk.link`

`RoomTarget` (`__init__.py`) holds this; `remote` is true when its `wsServer` is not the account's own.

6.2 Discovery while typing

`resolve_room()` uses the cached configuration, or discovers it (up to 12 s, `DISCOVERY_TIMEOUT`). The Join Conference panel starts the lookup 0.4 s after the typing stops, says "Looking up the SylkServer of …", keeps **Join disabled** until the answer, then shows "Conferencing provided by SylkServer at conference." or "No SylkServer found for : the room is joined over SIP". Join uses the answer it got.

6.3 Identity

The gateway authenticates nothing on `account-add` and `videoroom-join`: it checks the domain against its `sip_domains` (default `*`) and the room's allow/deny policy. Blink joins as **its own address** first; when the server answers `SIP domain not allowed` it joins once more as a **guest** of that server: `<display name> <4 hex>@<defaultGuestDomain>`, with a throwaway HA1, the display name kept.

6.4 Chat with a foreign room

The MSRP chat goes over SIP to `room@conference.<domain>`, through our own proxy and the other domain's. When it cannot connect (a proxy refusing to relay, an MSRP port that cannot be reached) the chat goes over the video room's WebRTC connection instead (§5.3).

6.5 What the foreign server needs

Found while making `domain.sylk.link` work (sylk-suite in Docker inside an LXC):

Problem	Symptom in Blink	Fix
Its OpenSIPS refused INVITEs from other domains to its conference domain	`403 Relaying denied`	`opensips-config-sylkserver`: `CONFERENCE_SERVER_PUBLIC` (default 1) lets requests to `CONFERENCE_SERVER_PREFIX.<local domain>` through the relay check
SylkServer advertised its container address in the MSRP path	`media stream timed-out while starting`; `[msrp] … advertised the private address 172.18.0.10:…`	`[MSRP] msrp_hostname` set to the public name (sylk-suite entrypoint: `FULL_DOMAIN`)
SylkServer listened on a random port per session	same, once the host was right	`sylkserver`: `[MSRP] port` (default 2856), one listening socket for all incoming sessions, each connection given to its session by the To-Path session id; published by Docker and forwarded by the LXC host (`lxc config device add … proxy listen=tcp:<public ip>:2856 connect=tcp:0.0.0.0:2856 nat=true`)
The self-signed certificate of the SylkServer package	`peer certificate signer not found … for CN Sylkserver issued by AG Projects Development`	sylk-suite: the certbot certificate mounted into the container, joined into the one file SylkServer takes and set as `certificate`
The certbot certificate had expired (renewals never ran)	`… EXPIRED` in SylkServer's log	the certbot container recreated with the fixed command, renewed by hand
Its OpenSIPS rebuilt the Contact of INVITE replies from the URI only	no `isfocus`: no conference-info, no participants list	`opensips-config-sylkserver`: the Contact is rewritten only to strip `pn-*` URI parameters, keeping its header parameters (`isfocus`)

Problem	Symptom in Blink	Fix
`sylkserver` from source needs `python3-sipsimple >= 5.6.0`	—	image rebuilt with `--no-cache` (5.6.4 from the repository)

Blink explains MSRP failures in the Activity and MSRP logs (`[msrp]`, `NOTE`): a private address in the path, no answer within the timeout, or a certificate refused, with the address and the certificate's subject and issuer.

6.6 Our side

Calling a foreign room's MSRP switch also needs **our** proxy to send the INVITE out for a local user; `403 Relaying denied` from our own proxy is a policy there, not in Blink.

7. Tile layout

For n tiles, every column count 1…n is tried, rows = ⌈n / columns⌉; each cell is the window minus 6 px gaps; the column count where the largest 16:9 rectangle in a cell has the largest area wins. Tiles take the whole cell, pictures are fitted inside. Ours is first, the others in joining order. In a 16:9 window: 2 side by side, 3–4 as 2×2, 5–6 as 3×2, 7–9 as 3×3.

8. Logging

Tag	What
`[webrtc]`	availability, the lookup of a foreign domain
`[sylkrtc]`	connections, accounts added, failed requests
`[videoroom]`	join, offer and answer (summary; full SDP at debug level), codecs, participants, ICE states, GStreamer errors, mute, camera
`[conference]`	joins, foreign room addresses, invitations
`[msrp]`	why an MSRP stream did not start

9. Limits and open items

- Outgoing video is a fixed 640×480 at 24 fps, letterboxed.
- Audio uses the system's default devices, not Blink's audio device settings.
- No composing indications or file transfers over WebRTC; no camera choice in the video window.
- An MSRP session that already has SIP audio when the video room starts keeps it.
- SylkServer's shared MSRP port keeps one stale entry when an outgoing actpass session is switched to a connector by sipsimple.
- After a certbot renewal SylkServer takes the new certificate only when restarted.